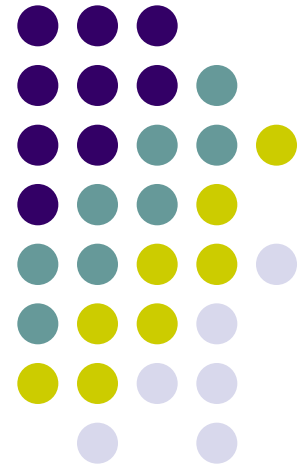


Bases de datos

Tema III – El lenguaje estructurado de consulta - SQL



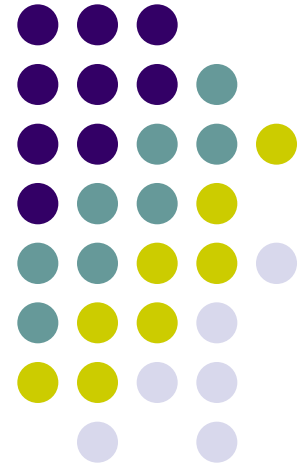
Universidad Nacional del
**FACULTAD DE INGEN
Y CIENCIAS HÍDRIC**



SQL *Structured Query Language*

Componentes del RDBMS

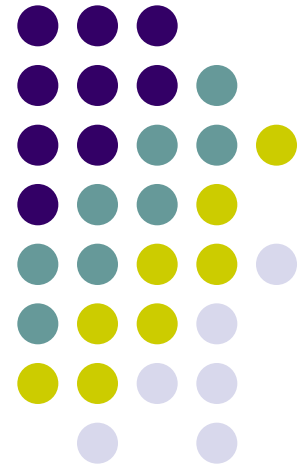
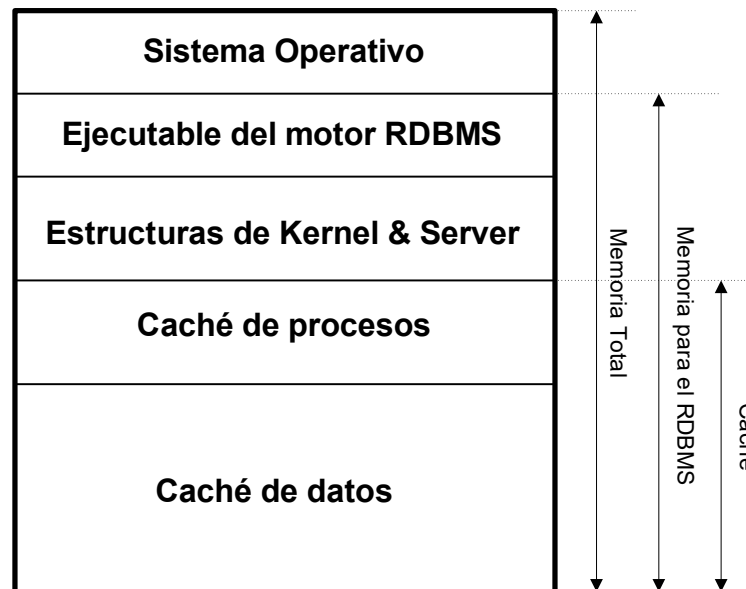
- **Dispositivos**
 - Físicos
 - Lógicos



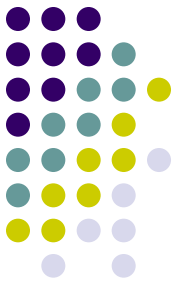
SQL *Structured Query Language*

Componentes del RDBMS

- **Dispositivos**
- **Memoria**



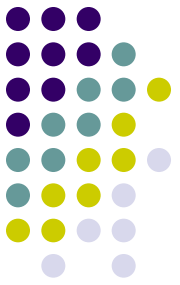
SQL *Structured Query Language*



Componentes del SQL

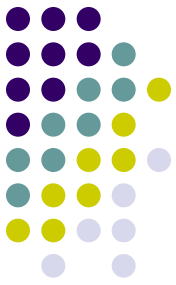
- Catálogo o diccionario de datos
- Los comandos
- Las palabras reservadas
- Los conectores lógicos y predicados
- El lenguaje de definición de datos
- El lenguaje de manejo de datos
- El lenguaje de control de datos

SQL *Structured Query Language*



Comandos del SQL

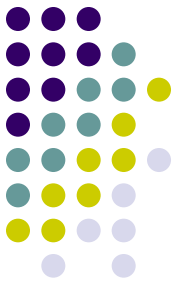
ALTER TABLE
BEGIN TRANSACTION
COMMIT TRANSACTION
CREATE
DELETE
DROP
GRANT
INSERT
REVOKE
ROLLBACK
SELECT
UPDATE



Las palabras reservadas

Son las palabras claves que forman los comandos de SQL, así como a las palabras calificativas que se usan con ellos.

No pueden utilizarse como identificadores o sea que no se pueden usar como nombres de tablas, de vistas o de columnas sin aplicarles símbolos especiales definidos por el implementador, con el fin de que se distingan de sus correspondientes palabras claves.



Tipos de datos y operadores

SQL funciona con **tres** tipos principales datos:

Cadenas alfanuméricas (CHAR, CHARACTER VARYING)

Numérico

Entero

Cortos (byte, smallint, short)

Largos (integer, bigint, numeric)

Decimal

Fecha (date, datetime, timestamp...)

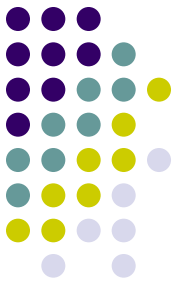
Operaciones básicas de valores:

Suma (+)

Resta (-)

Multiplicación (*)

División (/)



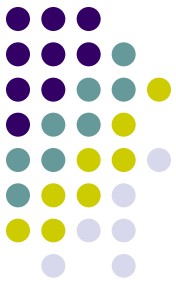
Lenguaje de definición de datos

Es aquel que sus instrucciones interactúan con el diccionario de datos manipulando su contenido agregando, modificando o eliminando su información. Los comandos son:

CREATE ...

ALTER ...

DROP ...



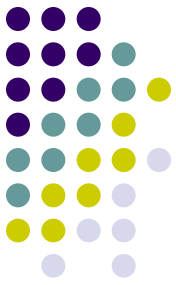
Lenguaje de definición de datos

CREATE DATABASE nombre_de_la_base [adicionales]

Dentro de adicionales, pueden ir distintos elementos que dependerán del motor sobre el que se ejecute la orden. Estos elementos pueden ser:

- si llevará o no archivo de *log*;
- espacio o dispositivo donde se alojarán los datos,
- espacio o dispositivo donde se alojará el log,
- espacio inicial tanto para datos como para log,
- forma de crecimiento de los dispositivos,
- etc

CREATE SCHEMA nombre_del_esquema



Lenguaje de definición de datos

CREATE DATABASE alumnado;

CREATE SCHEMA persona;

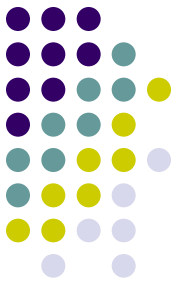
/ tablas provincia, localidad, persona, alumno, docente */*

CREATE SCHEMA estructura;

/ tablas departamento, materia, cargo */*

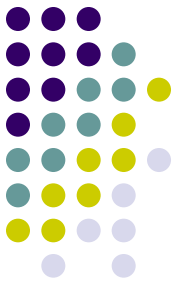
CREATE SCHEMA gestion;

/ tablas historialCargo, cursado, docenteCursado, detalleCursado */*



Lenguaje de definición de datos

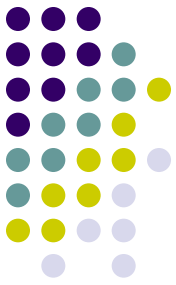
```
CREATE TABLE [esquema.]nombre_tabla (  
    Columna1 tipo_dato (tamaño)  
        [NOT NULL] [DEFAULT valor_por_default]  
        [CONSTRAINT nombre_constraint_de_columna]  
        [UNIQUE] [PRIMARY KEY]  
        [REFERENCES [esquema.]nombre_tabla (columna/s)]  
        [CHECK (condición)],  
    ....., ColumnaN.....,  
        [CONSTRAINT nombre_constraint_de_tabla]  
        [UNIQUE (columna/s)]  
        [PRIMARY KEY (columna/s)]  
        [CHECK (condición)]  
        [FOREIGN KEY (columna/s) REFERENCES [esquema.]nombre_tabla  
        (columna/s)]  
);
```



Lenguaje de definición de datos

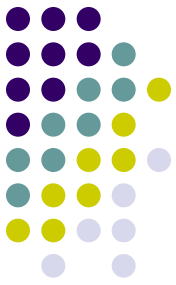
```
CREATE TABLE persona.provincia (  
  codigoprovincia smallint not null,  
  nombreprovincia varchar(50) not null);
```

```
CREATE TABLE persona.localidad (  
  codigoprovincia smallint not null,  
  codigolocalidad smallint not null,  
  codigopostal smallint not null,  
  nombrelocalidad varchar(50) not null);
```



Lenguaje de definición de datos

```
CREATE TABLE persona.persona (  
    tipodocumento      varchar(3)  not null,  
    nrodocumento        integer     not null,  
    codigoprovincianacio smallint    null,  
    codigolocalidadnacio smallint    null,  
    codigoprovinciavive  smallint    null,  
    codigolocalidadvive  smallint    null,  
    apellido            varchar(50) not null,  
    nombre               varchar(50) not null,  
    fechanacimiento      date        not null,  
    domicilio            varchar(50) null);
```



Lenguaje de definición de datos

Conceptos asociados

Páginas

Unidad básica de almacenamiento. Configurable.

Densidad de filas (fillfactor, max_row_per_page)

Espacio utilizable de una página dividido el tamaño de la fila

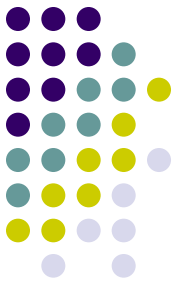
Porcentaje permitido para ocupación de una página

Máxima cantidad de filas por página

Extents

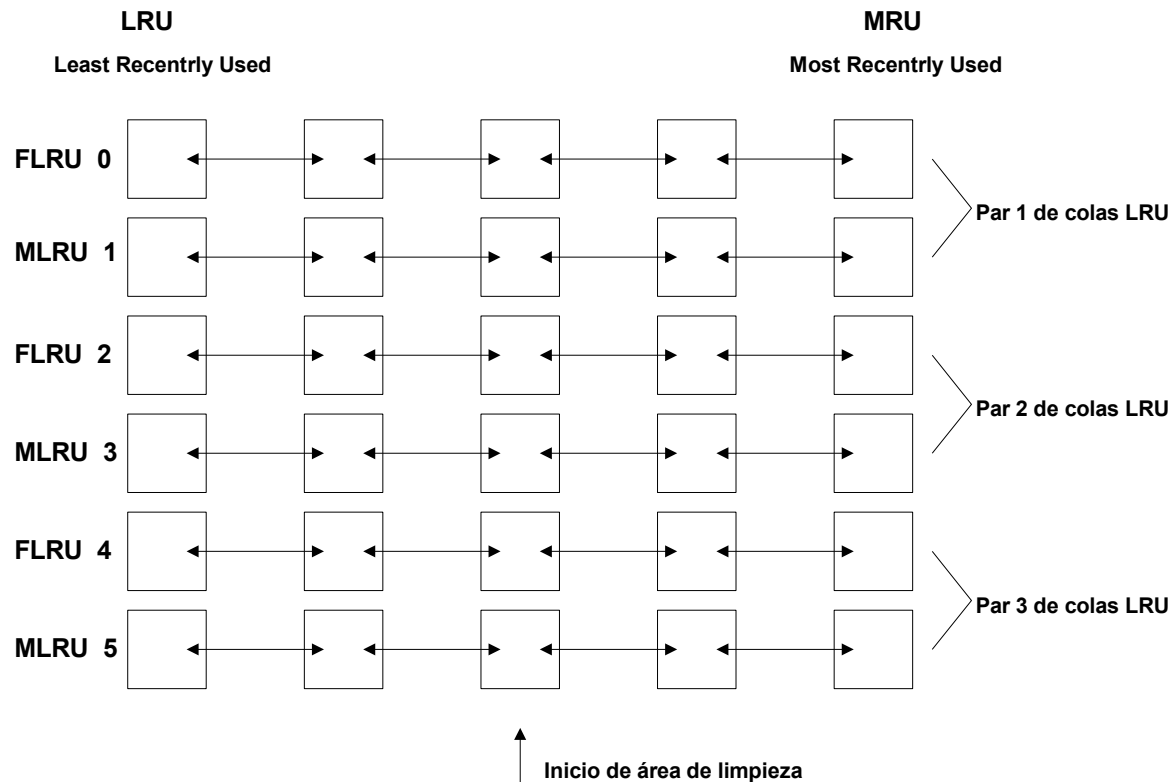
Espacio reservado simultáneamente para alojar objetos

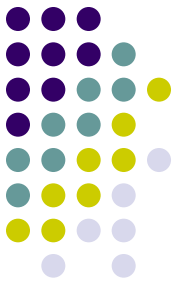
8 páginas



Lenguaje de definición de datos

Conceptos asociados





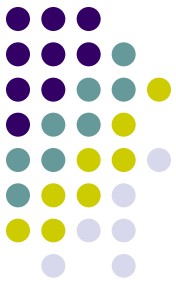
Lenguaje de manejo de datos

Inserción

```
INSERT INTO [esquema.]nombre_de_la_tabla  
    [(nombre_de_columna1,  
        nombre_de_columna2, ...)]  
VALUES ('valor1', 'valor2', ...);
```

```
INSERT INTO [esquema.]nombre_de_la_tabla  
    [(nombre_de_columna1,  
        nombre_de_columna2, ...)]  
(subconsulta);
```

Los valores que se inserten deberán ajustarse al tipo de datos de la columna en la que se van a insertar. Los tipos CHAR y DATE deben ir entre comillas o apóstrofes mientras que los valores numéricos y NULL simplemente se indican.



Lenguaje de manejo de datos

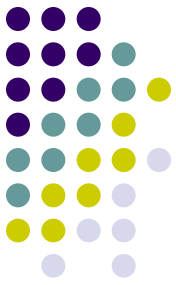
Inserción

```
INSERT INTO persona.provincia (codigoprovincia, nombreprovincia)  
VALUES (1, 'SANTA FE');
```

```
INSERT INTO persona.provincia VALUES (2, 'ENTRE RIOS');
```

```
INSERT INTO persona.localidad (codigoprovincia, codigolocalidad,  
codigopostal, nombrelocalidad) VALUES ( 1, 1,3000, 'SANTA FE');
```

```
INSERT INTO persona.localidad VALUES ( 2, 1, 3100, 'PARANA');
```



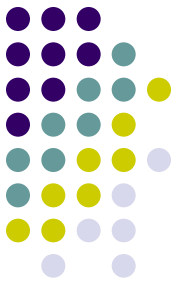
Lenguaje de manejo de datos

Actualización

```
UPDATE [esquema.]nombre_de_la_tabla  
    SET    columna1 = valor_nuevo,  
           columna2 = valor_nuevo,  
           ...  
           columnaN = valor_nuevo  
[WHERE condicion];
```

La cláusula SET indica las columnas que hay que actualizar y qué valores hay que poner en ellas. Se pueden actualizar múltiples columnas en cada fila, con un único comando UPDATE.

Actúa en todas las filas que satisfacen la condición especificada por la cláusula WHERE.



Lenguaje de manejo de datos

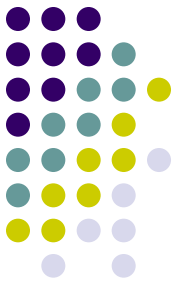
Actualización

Actualización por clave de negocio

```
UPDATE persona.localidad SET nombrelocalidad = 'PARANÁ'  
WHERE codigoprovincia = 2 AND codigolocalidad = 1;
```

Actualización por valor de atributo

```
UPDATE persona.localidad SET nombrelocalidad = 'PARANÁ'  
WHERE nombrelocalidad LIKE 'PARA%';
```



Lenguaje de manejo de datos

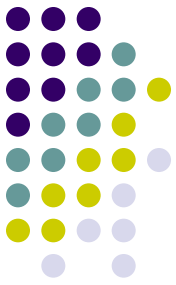
Borrado

***DELETE FROM [esquema.]nombre_de_la_tabla
[WHERE condición];***

No se pueden borrar parcialmente las filas.

Actúa en todas las filas que satisfacen la condición especificada por la cláusula WHERE.

El comando TRUNCATE TABLE elimina todos los datos de la tabla indicada sin dejar rastros en el log, y consecuentemente no pueden deshacerse los cambios (el borrado). Es más rápido pero peligroso.



Lenguaje de manejo de datos

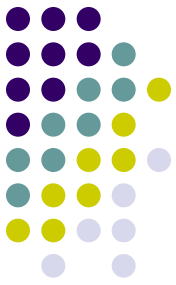
Borrado

Borrado por clave de negocio

```
DELETE FROM persona.localidad  
WHERE codigoprovincia = 2 AND codigolocalidad = 1;
```

Borrado por valor de atributo

```
DELETE FROM persona.localidad  
WHERE nombrelocalidad LIKE 'PARAN%';
```



Lenguaje de manejo de datos

Recuperación / Consulta

SELECT *columna1, columna2,,columnaN*

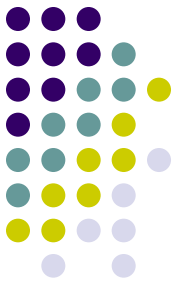
*FROM [esquema.]nombre_de_la_tabla1,...
 ,[esquema.]nombre_de_la_tablaN*

[WHERE condición]

[GROUP BY condición]

[HAVING condición]

[ORDER BY columna1 [ASC|DESC],...,columnaN [ASC|DESC];



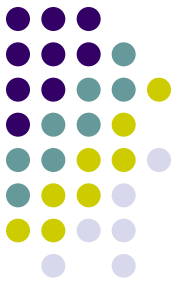
Lenguaje de manejo de datos

Recuperación / Consulta

SELECT * FROM persona.localidad;
*/*devuelve todas las columnas y filas */*

SELECT nombrelocalidad FROM persona.localidad
WHERE codigoprovincia = 1;
*/*devuelve el nombre de las localidades de la provincia 1*/*

SELECT apellido, nombre, tipodocumento, nrodocumento
FROM persona.persona
WHERE codigoprovinciavive = 1 AND codigolocalidadvive = 1;
*/*devuelve el apellido, nombre, tipo y número de documento de las personas que viven en la provincia 1 (SANTA FE) y localidad 1 (SANTA FE) */*

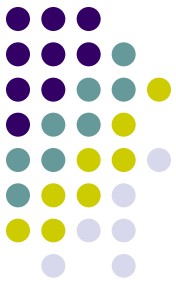


Lenguaje de definición de datos

Vistas

```
CREATE VIEW nombre_de_la_vista  
((nombre_columnas_de_la_vista))  
AS  
(SELECT columna(s) FROM [esquema.]tabla(s) WHERE condicion(es));
```

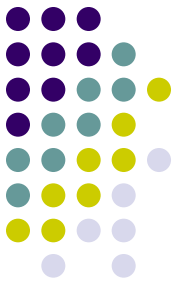
```
CREATE VIEW persona_santafe  
(tipodoc, nrodoc, ape, nom)  
AS (SELECT tipodocumento, nrodocumento, apellido, nombre  
FROM persona.persona  
WHERE codigoprovinciavive = 1 AND codigolocalidadvive = 1);  
/* vista con tipo y número de documento y apellido y nombre de las  
personas que viven en la provincial 1 y localidad 1) */
```

Lenguaje de definición de datos

Índices

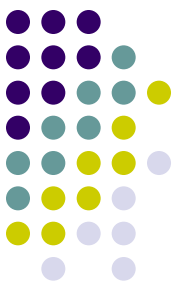
- **Aceleran la interacción con los datos de las tablas.**
- **Pueden crearse tantos índices como se deseen en una tabla cualquiera.**
- **Puede tener un índice para cada columna de la tabla, así como un índice para una combinación de columnas.**
- **Cuántos índices sean creados y de qué tipo para una determinada tabla, dependerán de los tipos de consultas esperadas que se dirigirán a la base de datos y además del tamaño de la misma.**
- **El crear demasiados índices puede presentar tantos inconvenientes como el crear muy pocos.**



Lenguaje de definición de datos

Índices

- **Mejoran el rendimiento reduciendo las entradas y salidas del disco. Realizan una función análoga a las de los índices de un libro acelerando la recuperación.**
- **Aseguran la unicidad. Puede crearse un índice único sobre una columna. Después, si se hace algún intento para insertar una fila que duplique el valor que ya está en esa columna, la inserción será rechazada. Por consiguiente, un UNIQUE INDEX actúa como una comprobación de la unicidad de la columna.**



SQL *Structured Query Language*

Lenguaje de definición de datos

Índices

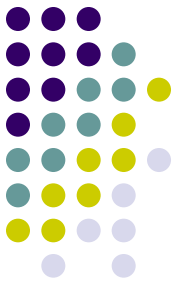
sqlserver

```
CREATE [UNIQUE] [CLUSTERED|NONCLUSTERED] INDEX  
    nombre_del_indice  
    ON [esquema.]nombre_de_la_tabla (nombre(s)_de_la(s)_columna(s));
```

postgresql

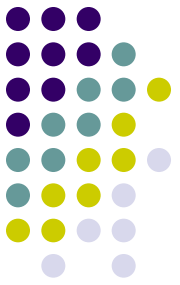
```
CREATE [UNIQUE] INDEX nombre_del_indice  
    ON [esquema.]nombre_de_la_tabla (nombre(s)_de_la(s)_columna(s))  
    [WHERE predicado];
```

```
CLUSTER [esquema.]nombre_de_la_tabla USING nombre_del_indice;
```



DDL – Integridad declarativa

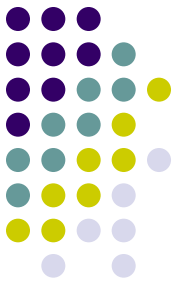
- La **Integridad Declarativa** es un mecanismo para condicionar e implementar la integridad de datos.
- Estas cláusulas se incorporan con las sentencias **create table** o **alter table**:
 - **Default** constraint
 - **Check** constraint
 - **Unique** constraint
 - **Primary key** constraint
 - **Reference** constraint
- Existen constraints de nivel columna y de nivel tabla.



DDL – Integridad declarativa

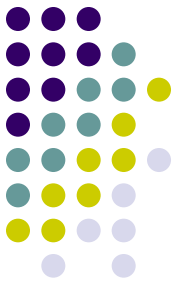
- La cláusula **default** pertenece al nivel columna.
- Una cláusula **default** es aplicada sobre un **insert** si no existe un valor que llene la columna.
- La cláusula **default** se utiliza para especificar el valor que ocupará la columna.
- Los **defaults** pueden ser constantes, null o *user* (char de 30 para el ultimo caso).
- Ejemplo:

```
CREATE TABLE persona.persona  
  (tipodocumento      varchar(3)   DEFAULT 'DNI' not null,  
   nrodocumento       integer      not null,  
   apellido            varchar(50) not null,  
   nombre              varchar(50) not null,  
   ...);
```



DDL – Integridad declarativa

- **Check constraints:**
 - Implementan integridad a los dominios.
 - Pueden ser aplicados a nivel columna o a nivel tabla.
- Son utilizados para especificar:
 - Una lista o conjunto de valores (a, b, c)
 - Un rango de valores (entre 0 y 255)
 - Un formato de edición (dos caracteres y un número: “AB3”)
 - Condiciones que debe cumplir un valor simple
- Es validado durante un **update** o **insert**.
- Especifica una “condición booleana”, restricción que el servidor de datos impone sobre todas las filas insertadas o modificadas en la tabla.

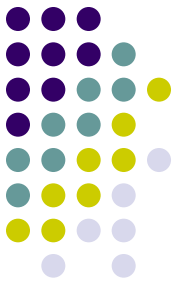


DDL – Integridad declarativa

Ejemplo (check de nivel columna):

```
CREATE TABLE persona.persona  
(tipodocumento varchar(3) not null  
    CONSTRAINT chk_tipodocumento  
    CHECK (tipodocumento IN ('DNI','LE','LC','PAS','OTR')),  
nrdocumento integer not null  
    CONSTRAINT chk_nrdocumento CHECK (nrdocumento > 1000000),  
apellido varchar(50) not null,  
nombre varchar(50) not null  
...;
```

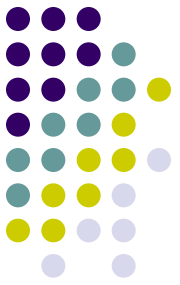
- Si el default viola la condición del check constraint, se rechazará cualquier inserción que use el valor default **pero éste será aceptado ante la ausencia de dato para la columna en cuestión.**



DDL – Integridad declarativa

Ejemplo (check de nivel tabla):

```
CREATE TABLE persona.persona  
(tipodocumento          varchar(3)  not null  
    CONSTRAINT chk_tipodocumento  
        CHECK (tipodocumento IN ('DNI','LE','LC','PAS','OTR')),  
nrodocumento            integer      not null  
    CONSTRAINT chk_nrodocumento CHECK (nrodocumento > 1000000),  
apellido                varchar(50) not null,  
nombre                  varchar(50) not null,  
fechanacimiento         date         not null,  
fechafallecimiento     date          null,  
    CONSTRAINT chk_fechanac_fechafalle  
        CHECK (fechanacimiento <= fechafallecimiento)  
);
```

DDL – Integridad declarativa

● Unique constraint

- Asegura que no haya dos filas que tengan el mismo valor de ese atributo o concatenación de atributos.
- Permite un solo valor NULL en la columna (en SQL server) y es irrelevante en Postgresql.
- Crea un **unique index nonclustered** por defecto

CREATE TABLE persona.alumno

(legajoalumno integer not null,

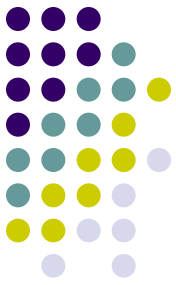
tipodocumento varchar(3) not null,

nrodocumento integer not null,

fechaingreso date not null,

CONSTRAINT unq_documento UNIQUE (nrodocumento, tipodocumento)

);

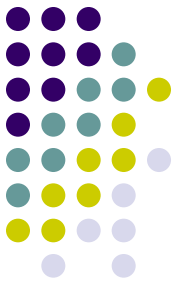


DDL – Integridad declarativa

- Primary Key constraint
 - Asegura que no existan dos filas con el mismo valor
 - No admite ningún valor NULL en la columna
 - Crea un unique index **por defecto**

Ejemplo (primary key a nivel columna):

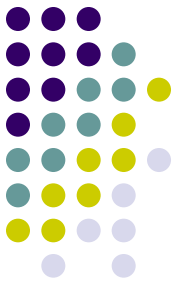
```
CREATE TABLE persona.alumno  
(legajoalumno      integer      not null PRIMARY KEY,  
tipodocumento    varchar(3)  not null,  
nrodocumento      integer      not null,  
CONSTRAINT unq_documento UNIQUE (nrodocumento,tipodocumento)  
);
```



DDL – Integridad declarativa

Ejemplo (primary key a nivel tabla):

```
CREATE TABLE persona.alumno  
(legajoalumno      integer      not null,  
tipodocumento    varchar(3)  not null,  
nrodocumento      integer      not null,  
CONSTRAINT unq_documento UNIQUE (nrodocumento, tipodocumento),  
CONSTRAINT pk_alumno PRIMARY KEY (legajoalumno)  
);
```



DDL – Integridad declarativa

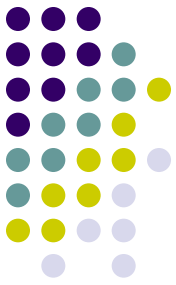
- Use **create table** para mantener la integridad referencial cuando una clave ajena es insertada o modificada o una clave primaria es borrada o actualizada.
- Nivel de columna:

```
CREATE TABLE [esquema].nombre_tabla (  
    columna tipo_dato [CONSTRAINT nombre_constraint]  
    REFERENCES [esquema.]tabla_referenciada  
    [(columna_referenciada)] ...
```

- Nivel de tabla:

```
[CONSTRAINT nombre_constraint]  
FOREIGN KEY (nombre_columna [{, nombre_columna}..])  
REFERENCES [esquema.]tabla_referenciada  
[(columna_referenciada [{, columna_referenciada}...])]
```

- Para claves compuestas se usan constraints de nivel de tabla.

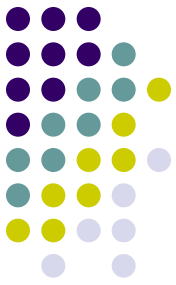


DDL – Integridad declarativa

Ejemplo (integridad referencial de nivel columna de ***localidad***):

- La tabla ***provincia*** debe existir y tener un índice unique sobre *codigoprovincia*
- Un valor de foreign key no puede ser insertado o modificado con un valor de *codigoprovincia* inexistente en *provincia*
- Un *codigoprovincia* en *provincia* no puede ser borrado o modificado si un valor de foreign key lo referencia

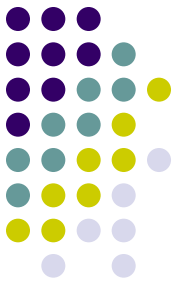
```
CREATE TABLE persona.localidad  
(codigoprovincia      smallint      not null,  
  REFERENCES persona.provincia (codigoprovincia),  
  codigolocalidad     smallint      not null,  
  codigopostal        smallint      not null,  
  nombrelocalidad     varchar(50)    not null  
CONSTRAINT pk_localidad  
  PRIMARY KEY (codigoprovincia, codigolocalidad));
```



DDL – Integridad declarativa

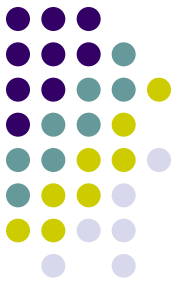
Ejemplo (integridad referencial a nivel tabla):

```
CREATE TABLE persona.persona  
(tipodocumento          varchar(3)  not null  
    CONSTRAINT chk_tipodocumento CHECK (tipodocumento IN ('DNI','LE','LC','PAS','OTR')),  
nrodocumento            integer      not null  
    CONSTRAINT chk_nrodocumento CHECK (nrodocumento > 1000000),  
codigoprovincianacio    smallint     null,  
codigolocalidadnacio    smallint     null,  
apellido                varchar(50)  not null,  
nombre                  varchar(50)  not null,  
fechanacimiento         date         not null,  
CONSTRAINT pk_persona PRIMARY KEY (tipodocumento, nrodocumento),  
CONSTRAINT fk_persona_locanacio  
    FOREIGN KEY (codigoprovincianacio, codigolocalidadnacio)  
    REFERENCES persona.localidad (codigoprovincia, codigolocalidad)  
);
```



DDL – Integridad declarativa

- **Alter Table** es usado para:
 - Agregar columnas a una tabla
 - Agregar o eliminar constraints de una tabla
 - Renombrar tablas
 - Renombrar columnas
 - Cambiar el tipo de dato
- La única manera de modificar una constraint es usando **alter table**
- Si una constraint es agregada o modificada o un default es reemplazado usando **alter table**, existiendo datos en la tabla, éstos no son afectados.



Lenguaje de definición de datos

Modificación de tablas

Para añadir otra columna, la sintaxis es:

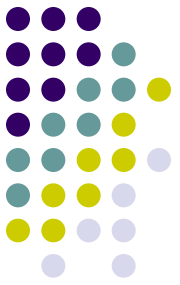
```
ALTER TABLE nombre_de_la_tabla  
ADD COLUMN nombre_de_la_columna tipo_de_datos;
```

Para cambiar el ancho de una columna ya existente, la sintaxis es:

```
ALTER TABLE nombre_de_la_tabla  
ALTER COLUMN nombre_de_la_columna  
TYPE tipo_de_datos nueva_anchura;
```

Para modificar el nombre de una columna:

```
ALTER TABLE nombre_tabla  
RENAME nombre_viejo TO nombre_nuevo;
```

Lenguaje de definición de datos

Modificación de tablas

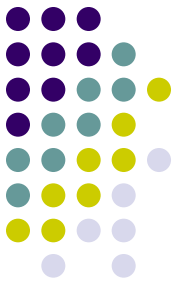
Para eliminar una columna:

```
ALTER TABLE nombre_de_la_tabla  
DROP COLUMN nombre_de_la_columna;
```

Para eliminar o agregar restricciones:

```
ALTER TABLE nombre_de_la_tabla  
ADD CONSTRAINT ... (igual definición que en la tabla);
```

```
ALTER TABLE nombre_de_la_tabla  
DROP CONSTRAINT nombre_constraint;
```



Lenguaje de definición de datos

Eliminación

La supresión de bases de datos:

DROP DATABASE nombre_de_la_base;

La supresión de tablas:

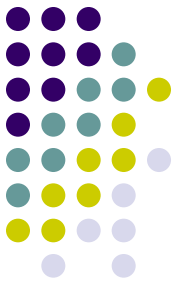
DROP TABLE nombre_de_la_tabla;

La supresión de vistas:

DROP VIEW nombre_de_la_vista;

La supresión de índices:

DROP INDEX nombre_del_índice ON nombre_de_la_tabla;

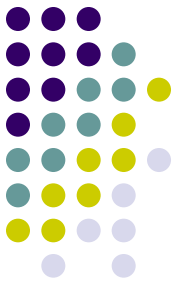


Lenguaje de control de datos

GRANT - REVOKE

Permiso	Objeto
SELECT	Tabla, Vista o Columna
UPDATE	Tabla, Vista o Columna
INSERT	Tabla, Vista
DELETE	Tabla, Vista
REFERENCES	Tabla, Columna
EXECUTE	Stored procedure

```
GRANT {ALL [PRIVILEGES] | lista_de_permisos }  
ON  
{  
    nombre_de_tabla [(lista_de_columnas)]  
    | nombre_de_vista [(lista_de_columnas)]  
    | nombre_procedimiento_almacenado  
}  
TO { PUBLIC | lista_de_usuarios | nombre_del_rol }  
[WITH GRANT OPTION]
```



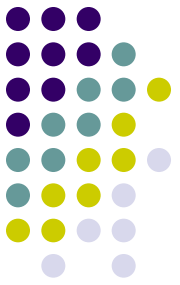
Lenguaje de control de datos

GRANT - REVOKE

Permiso	Objeto
SELECT	Tabla, Vista o Columna
UPDATE	Tabla, Vista o Columna
INSERT	Tabla, Vista
DELETE	Tabla, Vista
REFERENCES	Tabla, Columna
EXECUTE	Stored procedure

REVOKE

```
[GRANT OPTION FOR] {ALL [PRIVILEGES] | lista_de_permisos }  
ON  
  {  
    nombre_de_tabla [(lista_de_columnas)]  
    | nombre_de_vista [(lista_de_columnas)]  
    | nombre_procedimiento_almacenado  
  }  
FROM { PUBLIC | lista_de_usuarios | nombre_del_rol }
```



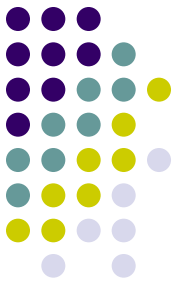
Lenguaje de control de datos

Transacciones

BEGIN TRANSACTION;

COMMIT TRANSACTION;

ROLLBACK TRANSACTION;



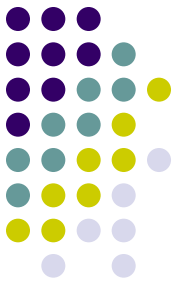
ACID (Atomicity, Consistency, Isolation and Durability)

Conjunto de características necesarias para que una serie de instrucciones puedan ser consideradas como una transacción

Atomicidad: es la propiedad que asegura que la operación se ha realizado o no, y por lo tanto ante un fallo del sistema no puede quedar a medias.

Consistencia: Integridad. Es la propiedad que asegura que sólo se empieza aquello que se puede acabar. Se ejecutan aquellas operaciones que no van a romper las restricciones de integridad de la base pasando desde un estado válido a otro también válido.

SQL *Structured Query Language*



ACID (**A**tomicity, **C**onsistency, **I**solation and **D**urability)

Aislamiento: una operación no puede afectar a otras. Dos transacciones sobre la misma información son independientes y no generan error.

Durabilidad: una vez realizada la operación, ésta persistirá y no se deshará aunque falle el sistema.

Cumpliendo estos 4 requisitos un sistema gestor de bases de datos puede ser considerado ACID Compliant.